

YOLOv8 추론 & Kalman Filter 개요

Day5 · 블록2

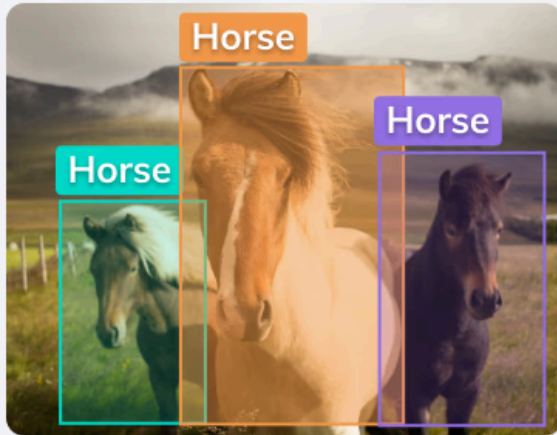
cchyun@gmail.com

Part 1. 객체 탐지와 YOLOv8 개념

객체 탐지(Object Detection)란?

- 로봇이 세상을 이해하기 위한 핵심 기술 — "로봇의 눈"
- 이미지 분류(Classification)
 - "이 사진 속에 무엇이 있는가?"에 집중
 - 예: "이것은 '사람'이다" — 한 단어로 정의
- 객체 탐지(Object Detection)
 - "무엇이 어디에 있는가?" 를 동시에 해결
 - 사진 속 물체를 찾아내고 위치를 사각형 박스로 표시
- 로봇 관점에서의 차이
 - 분류: "앞에 장애물이 있다" (위치 모름)
 - 탐지: "장애물이 왼쪽 2미터 지점에 있다" (위치 + 종류)
 - 로봇 주행 시 탐지 정보가 훨씬 중요

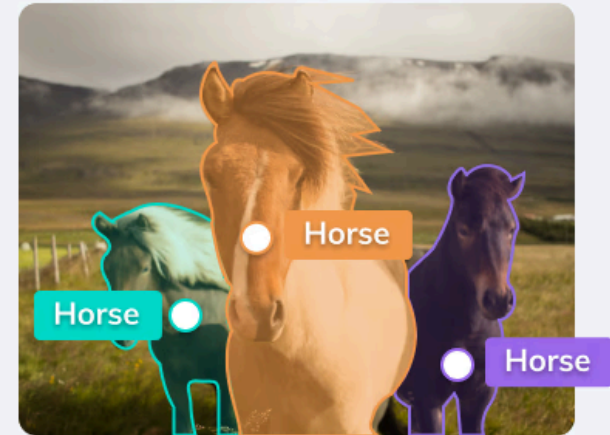
Object Detection + Semantic Segmentation = Instance Segmentation



Object detection



Semantic Segmentation



Instance Segmentation

V7 Labs

객체 탐지 모델 구조: 1-Stage vs 2-Stage

- 탐지 방식에 따라 두 가지 계열로 구분

구분	1-Stage (단일 단계)	2-Stage (두 단계)
대표 모델	YOLO, SSD, RetinaNet	Faster R-CNN, Mask R-CNN
동작 방식	위치 + 분류를 한 번에 예측	후보 영역 추출 → 분류 순차 처리
속도	빠름 (실시간 가능)	느림 (배치 처리에 적합)
정확도	상대적으로 낮음	상대적으로 높음
주요 용도	로봇, 자율주행, 엣지 장치	의료 영상, 정밀 검사

- 2-Stage 방식의 두 단계
 - 1단계 — RPN (Region Proposal Network)
 - 물체가 있을 법한 후보 영역(Anchor Box)을 수천 개 제안
 - 2단계 — Classification
 - 후보 영역 각각을 분류 → 높은 정확도, 높은 연산 비용
- 로봇 개발에서 1-Stage를 선택하는 이유
 - 실시간 제어 루프(30 FPS 이상)에서 지연 시간 최소화가 핵심
 - 라즈베리 파이 등 엣지 장치에서 2-Stage는 사실상 실시간 불가

YOLO의 핵심 아이디어: "You Only Look Once"

- YOLO(올로): "단 한 번만 본다"
 - 기존 방식의 문제점
 - 이미지에서 물체가 있을 법한 곳을 수천 번 훑어본 뒤 판단
 - 처리 속도가 느려 실시간 적용 어려움
 - YOLO의 혁신적 방식
 - 이미지를 격자(Grid)로 나눈 뒤
 - 신경망이 이미지 전체를 단 한 번의 평가(Evaluation) 만으로
 - 모든 사각형 박스와 물체 종류를 동시에 예측
 - 핵심 장점: 매우 빠름
 - 로봇처럼 실시간 즉각 판단이 필요한 장치에 최적화

- 2015년 등장 이후 다양한 팀이 발전시켜 온 실시간 탐지의 표준

버전	연도	개발	핵심 변화
v1	2015	Redmon et al.	Grid 기반 1-Stage 탐지, 45 FPS 실시간 처음 구현
v2	2016	Redmon et al.	Anchor Box 도입, Batch Normalization, Darknet-19
v3	2018	Redmon et al.	다중 스케일 탐지(3단계), Darknet-53, 소형 객체 성능↑
v4	2020	Bochkovskiy et al.	Bag of Freebies, IoU 손실 함수, CSPNet 백본
v5	2020	Ultralytics	최초 PyTorch 구현, 자동 Anchor 학습, 생산 환경 최적화
v8	2023	Ultralytics	Anchor-free, C2f 블록, 탐지·분할·자세추정 통합
v9	2024	Wang et al.	PGI(기울기 정보 보존), GELAN 아키텍처
v10	2024	칭화대	NMS-free 추론, One-to-One 헤드 구조
v11	2024	Ultralytics	C3k2·C2PSA 블록, 소형 객체 정확도↑, 엣지 최적화
v12	2025	Tian et al.	Attention 중심 설계(Area Attention), NMS-free, 낮은 지연·높은 mAP

- 1세대 — 개념 정립 (v1 ~ v3, 2015~2018)
 - Joseph Redmon이 설계한 원조 YOLO
 - "이미지를 단 한 번만 본다"는 1-Stage 패러다임 확립
 - v3에서 다중 스케일 탐지로 소형 객체 한계 극복
- 2세대 — 커뮤니티 주도 (v4 ~ v7, 2020~2022)
 - Redmon이 군사 적용 우려로 연구 중단 → 커뮤니티가 이어받음
 - Ultralytics의 v5가 PyTorch 기반으로 대중화에 결정적 기여
 - v4~v7은 정확도·속도 미세 최적화에 집중
- 3세대 — 다목적 통합 (v8 ~ v12, 2023~2025)
 - 탐지 단일 모델 → 탐지 + 분할 + 자세 추정 + 분류 단일 코드베이스
 - Anchor-free 설계가 표준으로 정착
 - v10·v11에서 NMS-free 구조로 엣지 장치 지연 시간 최소화
 - v12(2025.02)는 Attention 중심 아키텍처로 전환 — Area Attention 모듈로 기존 CNN 위주 설계에서 벗어나 정확도·속도 동시 개선

- YOLOv8

- 정확도와 속도 향상

- 더 정교한 설계(C2f 블록 등)
- 이전보다 적은 계산량으로 더 정확한 물체 감지

- 사용자 친화적 (Python-native)

- 복잡한 설정 파일 없이 Python 코드 몇 줄로 바로 실행
- 입문자도 쉽게 시작 가능

- 다양한 기능 — 단일 코드베이스

- 탐지(Detection): 물체의 위치와 종류 파악
- 분할(Segmentation): 물체의 형태를 픽셀 단위로 추출
- 자세 추정(Pose): 사람의 관절 위치 파악

Part 2. YOLOv8 설치 및 기본 사용법

설치 — 명령어 하나로 완료

```
pip install torch torchvision --index-url https://download.pytorch.org/whl/cpu
pip install ultralytics numpy==1.26.4 opencv-python==4.10.0.84
```

YOLOv8 실행에 필요한 모든 라이브러리가 설치됨

사전 학습 모델(Pre-trained Models) — 로봇 성능에 맞춰 선택

종류	특징	권장 환경
n (Nano)	가장 빠르고 가벼움	라즈베리 파이, 젯슨 나노 등 소형 로봇
s (Small)	속도와 정확도의 균형	일반적인 자율주행 로봇
m (Medium)	높은 정확도	고성능 PC 탑재 산업용 로봇
l/x (Large/X-Large)	최고 정확도, 느림	연구용, 복잡한 장면 분석

- 로봇 주행 시나리오(사람/장애물 탐지) 기본 코드

```
from ultralytics import YOLO
import cv2

# 1. 모델 로드 (가장 가벼운 Nano 모델 사용)
model = YOLO('yolov8n.pt')

# 2. 이미지 파일에서 추론
results = model.predict(source='robot_view.jpg', save=True)

# 3. 실시간 카메라(웹캠) 스트리밍 추론 (로봇 주행 시 사용)
# source=0은 기본 카메라를 의미
model.predict(source=0, show=True)
```

- `source`에는 이미지 파일 경로, 동영상 파일, 실시간 카메라 번호 입력 가능

- `results` 객체에서 핵심 데이터 추출
 - Bounding Box (xyxy)
 - 물체의 위치: `[x1, y1, x2, y2]` 형태
 - 왼쪽 상단과 오른쪽 하단의 좌표
 - 신뢰도(Confidence)
 - 0 ~ 1 사이의 값
 - "이것이 사람일 확률이 얼마나 높은가?"
 - 보통 0.5(50%) 이상일 때만 진짜로 인식
 - 클래스(Class)
 - 찾은 물체의 번호
 - 예: 0번 = '사람', 2번 = '자동차'

데이터 추출 코드 예시

```
results = model('robot_view.jpg')
for result in results:
    boxes = result.boxes # 상자 정보들
    for box in boxes:
        # 좌표 (좌상단 x, y, 우하단 x, y)
        x1, y1, x2, y2 = box.xyxy

        # 신뢰도 점수
        conf = box.conf

        # 클래스 번호
        cls_id = int(box.cls)

        # 물체 이름 확인 (예: person, bicycle...)
        label = model.names[cls_id]

    print(f"발견: {label}, 위치: ({x1}, {y1}), 확률: {conf:.2f}")
```

- 로봇 제어 로직 예시

- `label == 'person'` 이고 `conf > 0.5` 이면 → "긴급 정지" 명령

- 결과 시각화 — 코드 한 줄로 완료

```
annotated_frame = results.plot() # 박스, 이름, 확률 자동으로 그림
cv2.imshow("Robot Vision", annotated_frame)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

- 입문자가 자주 하는 실수

- CPU vs GPU 문제: GPU(NVIDIA) 없으면 수십 배 느림 — 설치 시 CUDA 버전 확인
- 경로 오류: 모델 파일/이미지 파일이 코드와 같은 폴더에 있어야 함
- 이미지 크기 미조정: `imgsz=320` 옵션으로 속도 향상 가능 (기본값 640)
- 카메라 번호 혼동: 카메라가 여러 대라면 `source=0` 이 아닌 `1`, `2` 일 수 있음

Part 3. YOLOv8 실전 활용 + 최적화

```
import cv2
from ultralytics import YOLO

model = YOLO('yolov8n.pt') # Nano 모델이 속도가 가장 빠름
cap = cv2.VideoCapture(0) # 0은 기본 웹캠

while cap.isOpened():
    success, frame = cap.read()
    if not success:
        break

    # stream=True 옵션은 메모리 효율적인 제너레이터를 반환
    results = model.predict(frame, stream=True)

    for r in results:
        annotated_frame = r.plot() # OpenCV 호환 배열 반환
        cv2.imshow("YOLOv8 Real-time Inference", annotated_frame)

    if cv2.waitKey(1) & 0xFF == ord('q'): # 'q' 키로 종료
        break

cap.release()
cv2.destroyAllWindows()
```

- Confidence Threshold 조절

- `model.predict(conf=0.5)` 로 낮은 확률의 거짓 양성(False Positive) 제거

- 특정 클래스만 탐지

- YOLOv8은 COCO 데이터셋의 80개 클래스 기본 지원
- `classes` 매개변수로 원하는 클래스 인덱스만 지정

```
model.predict(classes=[0, 2]) # 사람(0)과 자동차(2)만 탐지
```

- NMS (Non-Maximum Suppression)

- 하나의 객체에 여러 경계 상자 생성 시, 가장 높은 점수만 남기고 나머지 제거
- 각 상자 간의 **겹침 정도(IoU)** 를 계산하여 임계값 이상 겹치는 상자 삭제
- 최신 YOLO 버전(YOLO10 등)은 NMS-free 아키텍처 채택

프로젝트 요구사항에 따른 Trade-off

모델	속도 (GPU)	파라미터 수	주요 용도
YOLOv8n (Nano)	~1.0 ms	3.2M	모바일, 엣지 장치, 초고속 탐지
YOLOv8s (Small)	~2.5 ms	11.2M	일반적인 로봇 픽애플레이스
YOLOv8m (Medium)	~6.0 ms	25.9M	복잡한 산업 검사
YOLOv8x (X-Large)	~15.0 ms	68.2M	고정밀 연구, 복잡한 장면

추가 최적화 방법

- 이미지 해상도 조절: `imgsz` 낮추면 속도↑, 소형 객체 탐지 성능↓
- GPU 활용: NVIDIA CUDA 지원 GPU에서 수십 배 빠름 (실시간 로봇 필수)
- 엣지 최적화: Intel OpenVINO, ONNX, TensorRT 변환 + FP16/INT8 양자화

- 지연 시간(Latency) 누적
 - 문제: 처리가 입력 속도보다 느려 화면이 밀리는 현상
 - 해결: `model.predict(stream=True)` 로 제너레이터 방식 처리, 최신 프레임만 사용
- 탐지 결과의 흔들림(Flickering)
 - 문제: 경계 상자가 프레임마다 미세하게 떨리는 현상
 - 해결: 칼만 필터(Kalman Filter) 도입 — 이전 프레임 기반 예측 및 보정
- 엣지 장치(Raspberry Pi 등) 성능 저하
 - 문제: CPU 점유율 100%, 1~2 FPS 수준
 - 해결: 모델을 ONNX 또는 TensorRT 형식으로 내보내기(Export), 양자화 적용

- 로봇 시스템에서 탐지 결과를 다른 노드로 전달
 - `sensor_msgs/Image` 를 입력받아 `vision_msgs` 형태로 퍼블리시
- 추천 데이터 구조
 - **Header:** 타임스탬프 및 프레임 ID (TF2 좌표 변환에 필수)
 - **Detections (리스트):**
 - `class_id` (int): 객체 종류
 - `confidence` (float): 신뢰도
 - `bbox` (float[4]): [x_center, y_center, width, height] (픽셀 좌표)
- ROS2 통합 팁
 - `cv_bridge` 를 사용하여 ROS2 이미지 메시지 → OpenCV(NumPy 배열) 변환
 - 변환 후 YOLO 처리 수행

Part 4. Kalman Filter 개념 개요

- 모든 센서에는 '노이즈(Noise)'라는 방해꾼이 존재
 - GPS 센서 (측정치)
 - 로봇의 현재 위치를 알려주지만 정확도가 떨어짐
 - 실제 위치에서 10m 정도 왔다 갔다 → 이 데이터만 믿으면 **지그재그 이동**
 - 로봇의 계산 (예측치)
 - 바퀴 회전 수로 "나는 지금쯤 어디 있겠구나" 추측
 - 하지만 바퀴가 헛돌거나 바람에 밀릴 수 있어 **완벽하지 않음**
 - 칼만 필터의 역할
 - 불안정한 센서 데이터(GPS) + 불완전한 내 추측(바퀴 회전수)
 - → 적절히 융합하여 실제 위치에 가장 가까운 값을 실시간으로 계산

- 칼만 필터는 두 단계를 끊임없이 반복하는 순환 구조
 - 1단계: 예측 (Predict) — "어디로 갈 것인가?"
 - 현재 속도와 방향으로 다음 순간의 위치를 **미리 짐작**
 - 비유: 날씨 예보 — "어제 기압과 바람을 보니 내일은 비가 올 것 같아"
 - 시간이 지날수록 **불확실성(오차 범위)**이 커짐
 - 2단계: 업데이트 (Update/Correct) — "실제로는 어디인가?"
 - 새로운 센서 데이터가 들어오면 예측과 측정값을 비교·수정
 - 비유: 예보(비)였는데 창밖을 보니 구름만 → "흐림으로 수정"
- 칼만 이득(Kalman Gain)
 - 예측과 센서 중 무엇을 더 믿을지 결정하는 **가중치**
 - 센서가 정확하면 센서를 더 믿고, 센서가 튀면 예측을 더 믿음

- **물체 추적 (YOLO 탐지 결과 안정화)**
 - YOLO 경계 상자(Bounding Box)가 미세하게 떨리는 현상 발생
 - 칼만 필터로 움직임을 부드럽게(Smoothing)
 - 물체가 잠시 가려져도 다음 위치를 예측하여 추적 유지
- **위치 추정 (GPS + IMU 융합)**
 - GPS: 절대 위치 제공, 느림
 - IMU(가속도/자이로): 빠르지만 오차 누적
 - 칼만 필터로 두 센서를 융합 → **빠르고 정확한 위치 정보**
- **속도 추정**
 - 위치 센서값만 있을 때
 - 시간에 따른 위치 변화를 계산하여 **현재 속도를 추정**

EKF (Extended Kalman Filter)란?

- 기본 칼만 필터의 한계와 확장
 - 기본 칼만 필터의 가정
 - 물체가 직선으로 일정하게 움직이는 '선형 시스템' 가정
 - 곡선 이동이나 복잡한 움직임에는 적용 어려움
 - 현실의 로봇 — '비선형 시스템'
 - 커브를 돌거나 복잡한 움직임
 - 기본 칼만 필터를 직접 적용할 수 없음
 - EKF의 해결 방법
 - 복잡한 곡선 움직임을 순간순간 잘게 쪼개어
 - 직선처럼 취급(선형화)
 - → 칼만 필터의 원리를 비선형 시스템에도 적용 가능

Part 5. 칼만 필터 — 원리 깊이 이해하기

상태(State)란 무엇인가?

- 칼만 필터가 추적하는 것 — 위치 p 와 속도 v

$$\vec{x} = \begin{bmatrix} p \\ v \end{bmatrix}$$

- 왜 속도도 함께 추적하는가?

- 위치만 알아서는 "다음에 어디 있을지" 모름
- 속도를 알면 다음 위치를 **예측** 가능
- 비유: 친구 차가 지금 서울에 있다는 것만 알면 1시간 뒤 위치를 모른다.
하지만 "**서울에서 시속 100km로 부산 방향**" 이라면 예측 가능!

- 둘 다 정확한 값을 모른다 → 칼만 필터가 추정하는 대상

- 추정값의 **중심** = $\hat{x}_k$ (가장 그럴듯한 값)
- 추정값의 **불확실성** = P_k (얼마나 모르는지)

- 공분산 행렬 P_k — 2D 불확실성 표현

$$P_k = \begin{bmatrix} \sigma_p^2 & \sigma_{pv} \\ \sigma_{vp} & \sigma_v^2 \end{bmatrix}$$

원소	의미
σ_p^2 (대각)	위치를 얼마나 모르는가
σ_v^2 (대각)	속도를 얼마나 모르는가
σ_{pv} (비대각)	위치 오차와 속도 오차가 얼마나 연결되어 있는가

- P_k 는 대칭행렬 — $\sigma_{pv} = \sigma_{vp}$
 - "위치 오차와 속도 오차의 상관관계"는 어느 방향에서 봐도 동일

- 아래 첨자 k 의 의미 — 시간 스텝

- $k = 0, 1, 2, 3 \dots$ = "몇 번째 시간 순간인가" (예: 매 0.1초마다 1씩 증가)
- 한 스텝 안에서 $\text{Predict}(\hat{x}_k^-, P_k^-) \rightarrow \text{Update}(\hat{x}_k, P_k)$ 두 번 변함

```
k=1 진입 → Predict →  $\hat{x}_k^-$ ,  $P_k^-$    (불확실성 증가)
           → Update →  $\hat{x}_k$ ,  $P_k$    (불확실성 감소)
k=2 진입  ( $\hat{x}_k$ ,  $P_k$  가 다음 스텝의  $\hat{x}_{k-1}$ ,  $P_{k-1}$ 이 됨)
```

- 초기값 P_0 — 사용자가 직접 설정

- 실제 오차를 계산하는 게 아니라 **"얼마나 모른다고 가정할지"** 를 설계자가 결정
- 잘못 설정해도 몇 스텝 안에 현실적인 값으로 자동 수렴
- 불확실하면 **크게** 설정하는 것이 안전

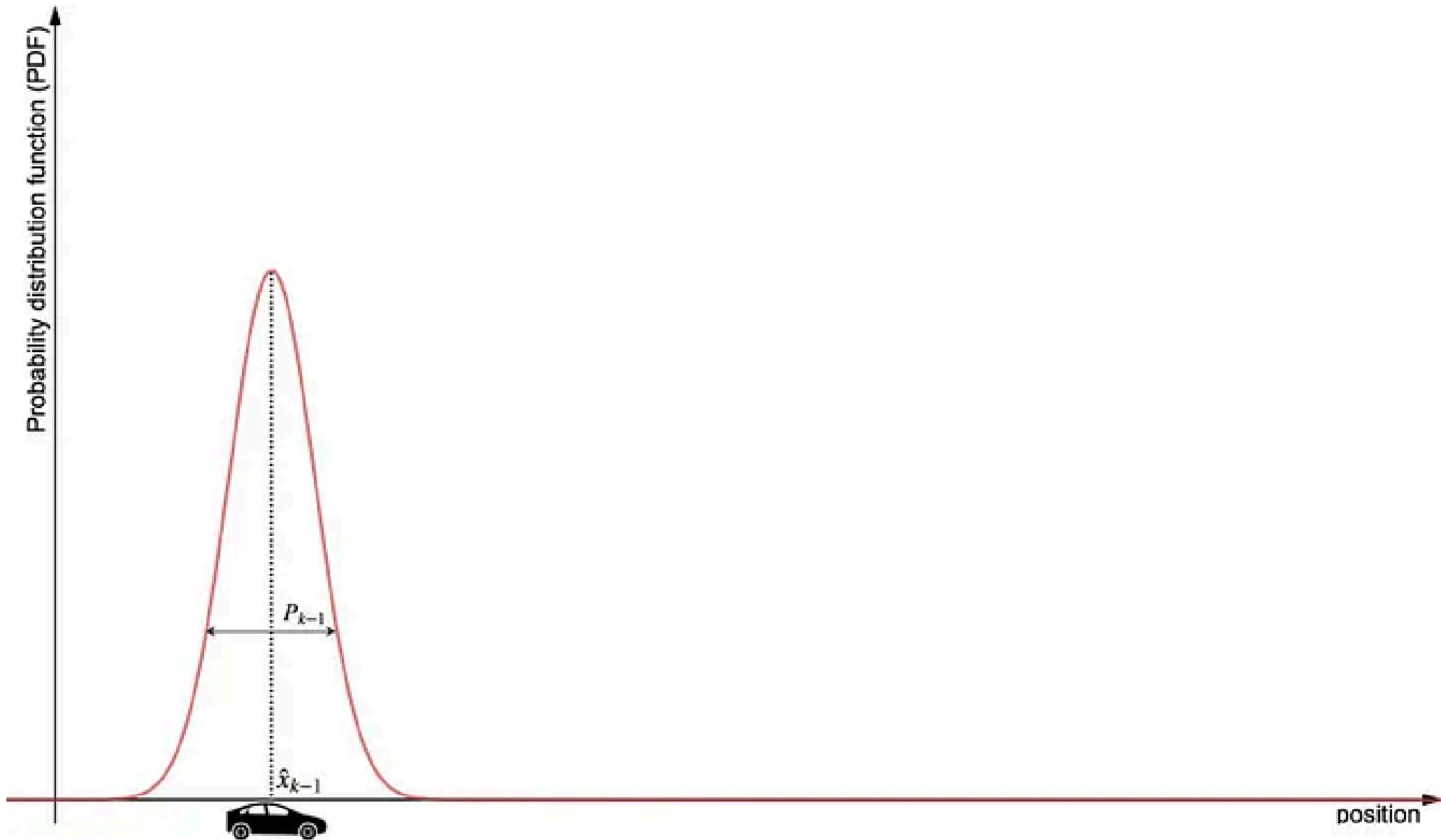
```
# "초기 위치는  $\pm 5m$ , 속도는  $\pm 2m/s$  정도로 불확실하다" 고 가정
# 분산 = 표준편차2 →  $5^2 = 25$ ,  $2^2 = 4$ 
P_0 = [[25, 0],
        [0, 4]]
```

노이즈 행렬 Q와 R — "무엇을 믿을 것인가"

	Q_k (프로세스 노이즈)	R_k (센서 노이즈)
의미	내 예측이 얼마나 틀릴 수 있는가	센서가 얼마나 부정확한가
비유	"바람이 세서 내 예측이 빗나갈 수 있어"	"GPS가 $\pm 10\text{m}$ 오차야"
적용 단계	Predict	Update
설정 방법	설계자 경험으로 설정 (어려움)	센서 데이터시트 참조 (비교적 쉬움)
크기	상태변수 수 $\times$ 상태변수 수	센서 수 $\times$ 센서 수

- R_k 크기는 센서 개수에 따라 결정됨
 - GPS만 있을 때 $\rightarrow 1 \times 1$ 스칼라
 - GPS + 속도계 $\rightarrow 2 \times 2$ 행렬
 - YOLO 중심좌표 $(x, y) \rightarrow 2 \times 2$ 행렬

```
# 센서를 고정하고 많이 측정  $\rightarrow$  분산 = 노이즈 크기
readings = gps.measure(n=1000)
R = np.var(readings) # GPS 노이즈 공분산 계산
```



$$\hat{x}_k^- = A\hat{x}_{k-1} + Bu_k$$

$$P_k^- = AP_{k-1}A^T + Q_k$$

- A — 상태 전이 행렬 (물리 법칙을 행렬로 표현)

- 등속 운동 가정: $p_k = p_{k-1} + v_{k-1} \cdot \Delta t$, $v_k = v_{k-1}$
- 등속 가정은 단순화 — 실제 로봇은 가속·감속하지만, 매우 짧은 Δt 안에서는 등속으로 볼 수 있어 유효함.
- 가속도는 Bu_k 로 별도 처리, 모르는 외란은 Q_k 가 흡수

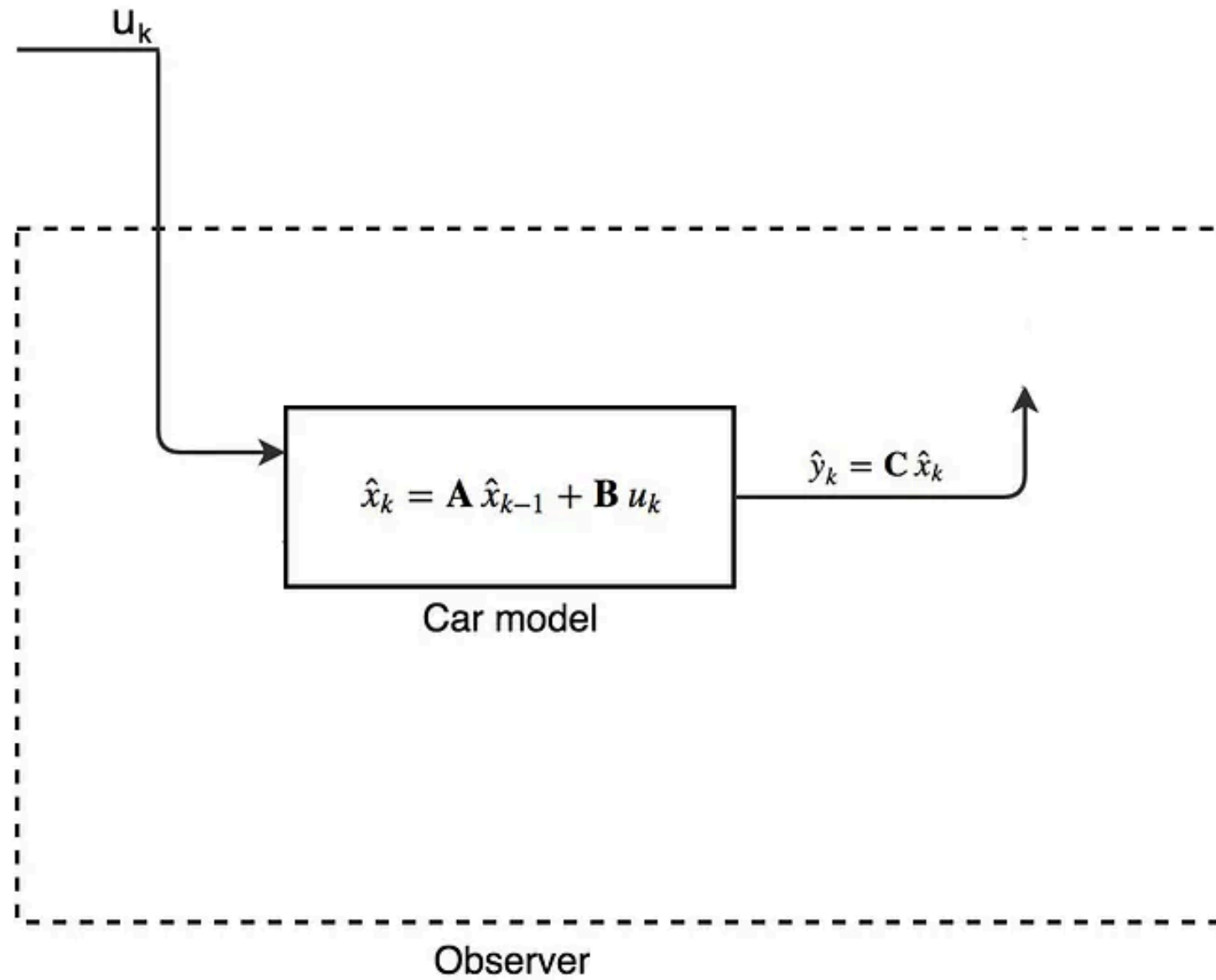
$$A = \begin{bmatrix} 1 & \Delta t \\ 0 & 1 \end{bmatrix}$$

- Bu_k — 제어 입력 (내가 내린 명령)

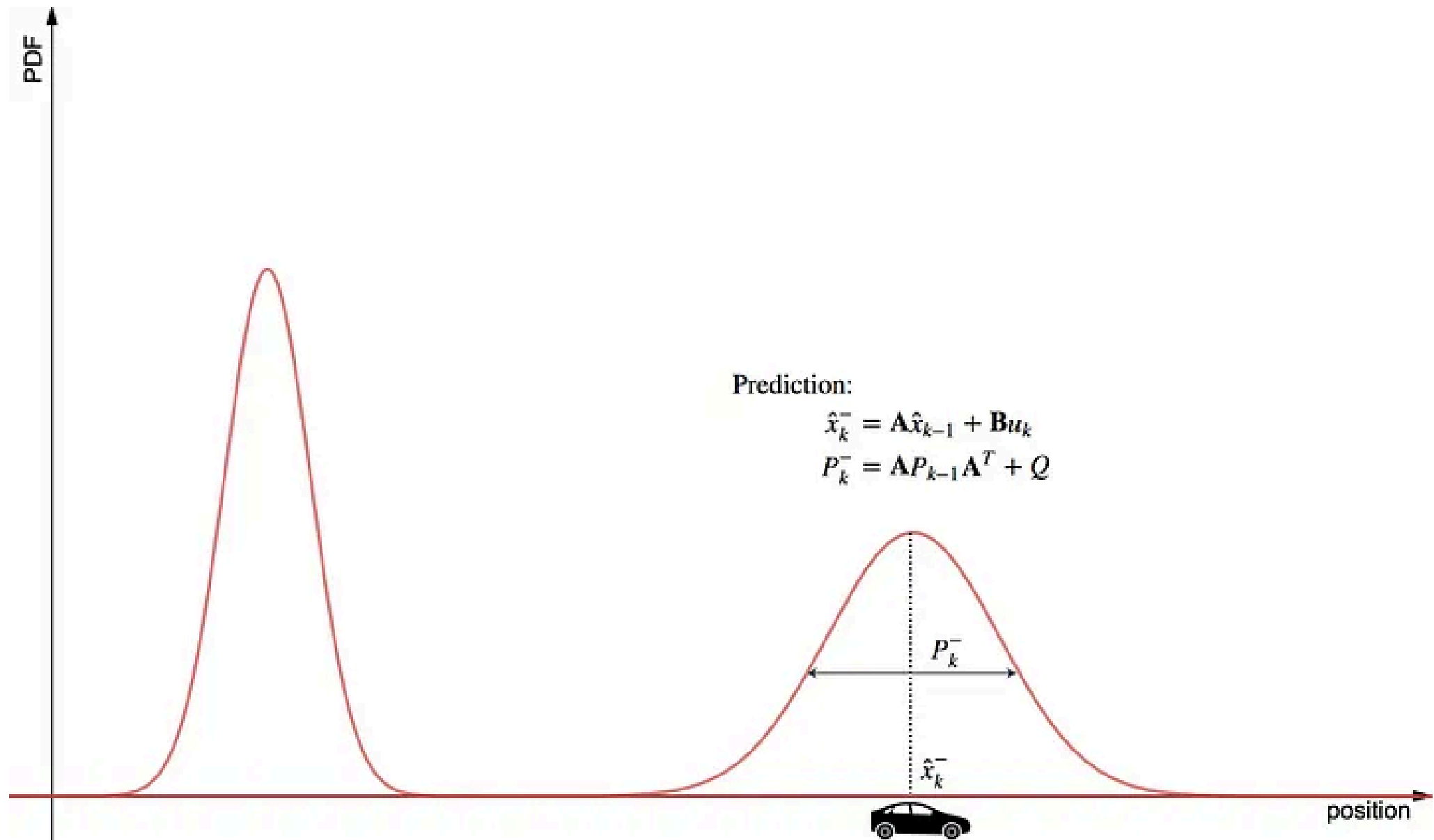
- 로봇에 가속도 명령 a 를 내렸다면:

$$p_k = p_{k-1} + v_{k-1}\Delta t + \frac{1}{2}a\Delta t^2, \quad v_k = v_{k-1} + a\Delta t$$

- 핵심: Predict 후 P_k^- 가 커짐 — 예측만으로는 점점 불확실해짐



Predict Result



- C — 센서 행렬: 상태 $\rightarrow$ 측정 공간으로의 투영
 - 상태벡터 $[p, v]^T$ 중 센서가 측정 가능한 성분만 추출하는 행렬
 - C 의 1 = 해당 상태변수를 측정 가능, 0 = 측정 불가

$$C = \begin{bmatrix} 1 & 0 \end{bmatrix}$$

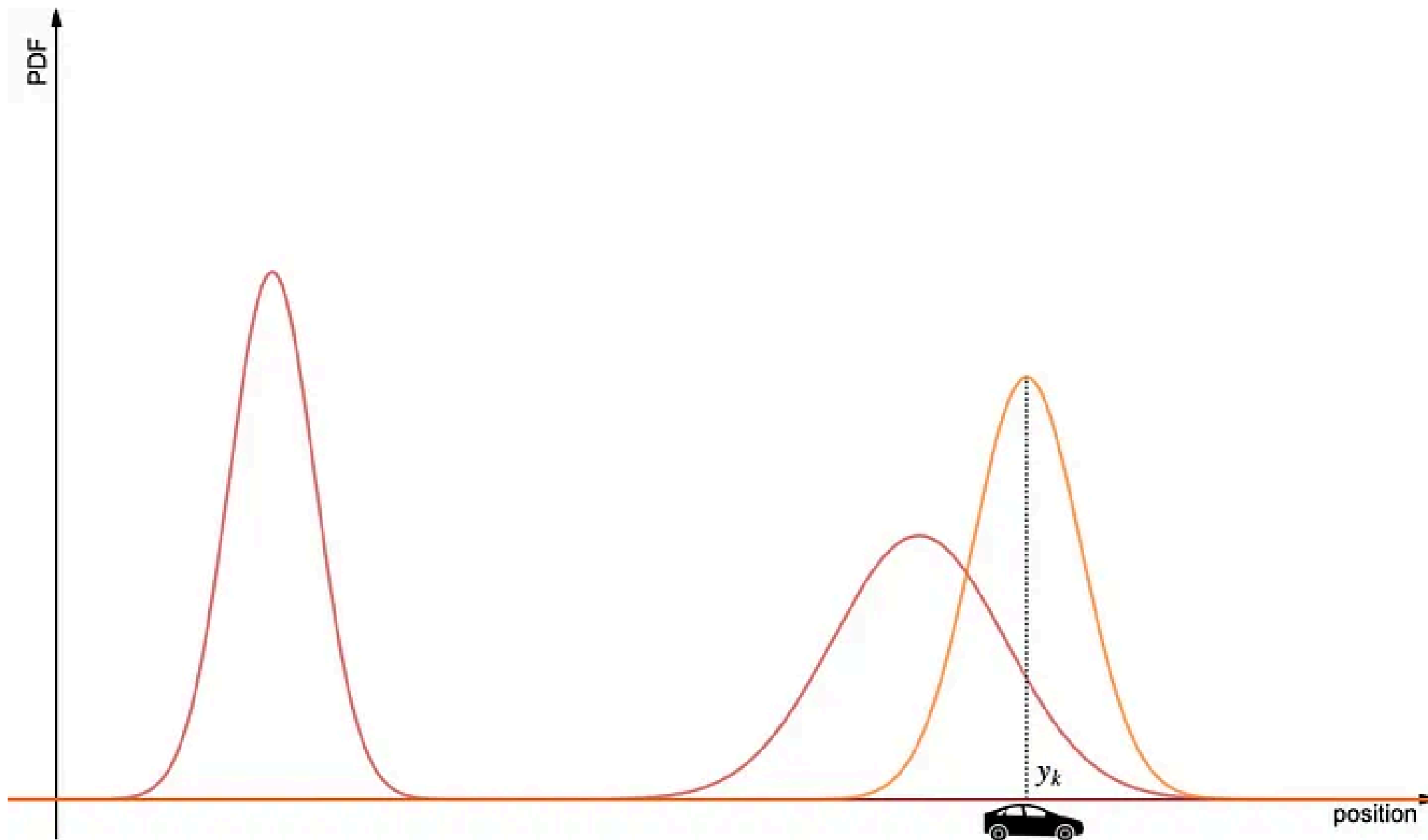
- 예측 측정값 vs 실제 측정값

$$\hat{y}_k = C \hat{x}_k^- = \begin{bmatrix} 1 & 0 \end{bmatrix} \begin{bmatrix} p \\ v \end{bmatrix} = p \quad \leftarrow \text{모델이 예측한 측정값}$$

$$y_k \quad \leftarrow \text{실제 GPS 센서 측정값}$$

- 둘은 다르다 $\rightarrow$ 그 차이가 Innovation

$$\text{Innovation} = y_k - \hat{y}_k = y_k - C \hat{x}_k^-$$



Update 단계 — "칼만 게인"

- 칼만 게인 K_k — 예측 vs 센서, 무엇을 더 믿을까?
- 행렬에서는 나눗셈이 없음 — $(\dots)^{-1}$ 은 역행렬을 오른쪽에 곱하는 것

$$K_k = P_k^- C^T \cdot \underbrace{(C P_k^- C^T + R_k)^{-1}}_{\text{역행렬}}$$

상황	K_k 값	의미
예측이 매우 불확실 ($P^- \gg R$)	$K_k \rightarrow 1$	센서를 거의 100% 신뢰
센서가 매우 부정확 ($P^- \ll R$)	$K_k \rightarrow 0$	예측을 거의 100% 신뢰
비슷한 수준	$K_k \approx 0.5$	반반

$$\hat{x}_k = \hat{x}_k^- + K_k(y_k - C\hat{x}_k^-)$$

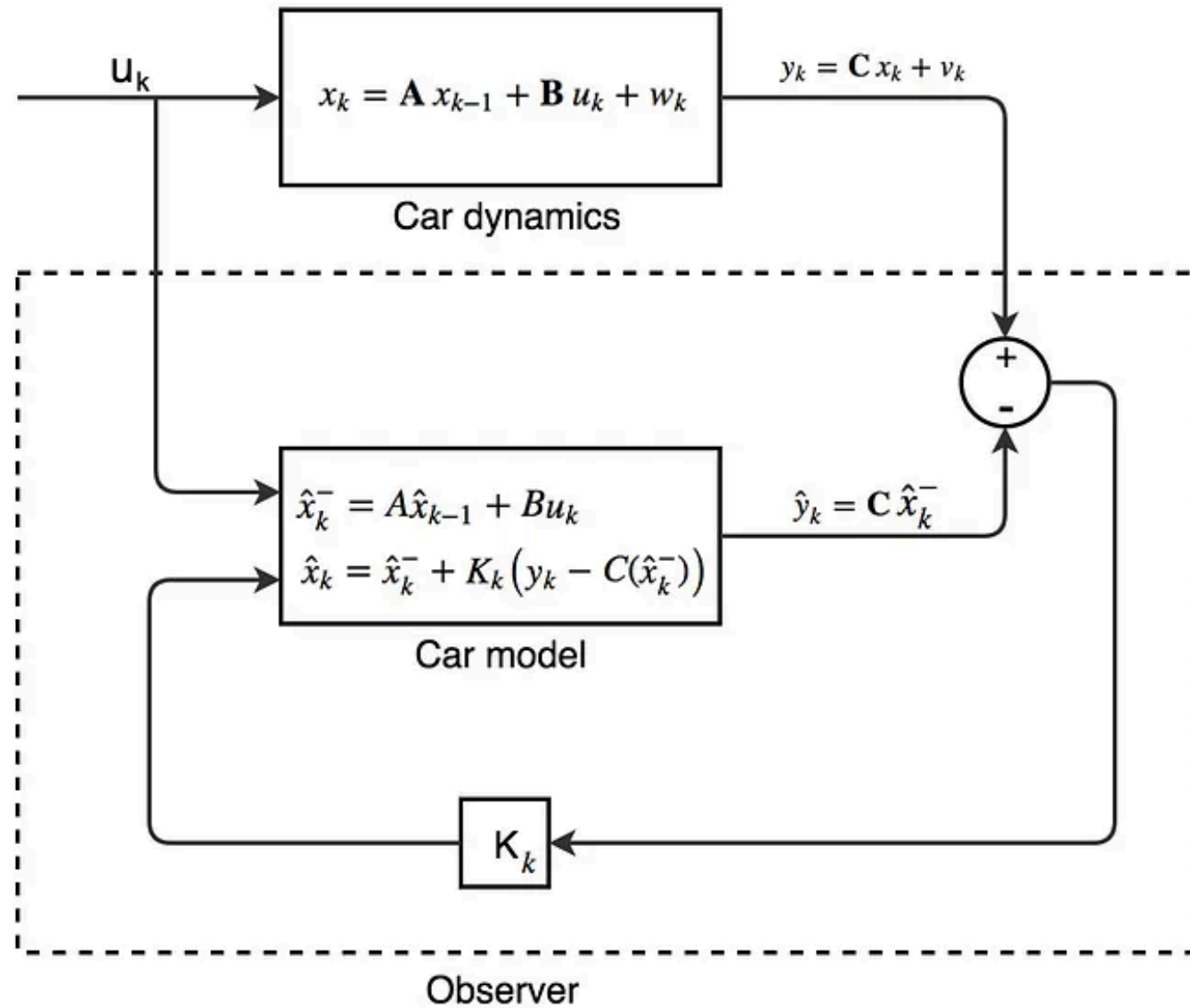
$$P_k = (I - K_k C)P_k^-$$

항	이름	의미
$y_k - C\hat{x}_k^-$	Innovation	예측과 실제 측정의 차이
$K_k \times \text{Innovation}$	보정량	이 차이 중 얼마나 반영할지
$(I - K_k C)P_k^-$	불확실성 감소	K_k 비율만큼 P^- 를 줄임

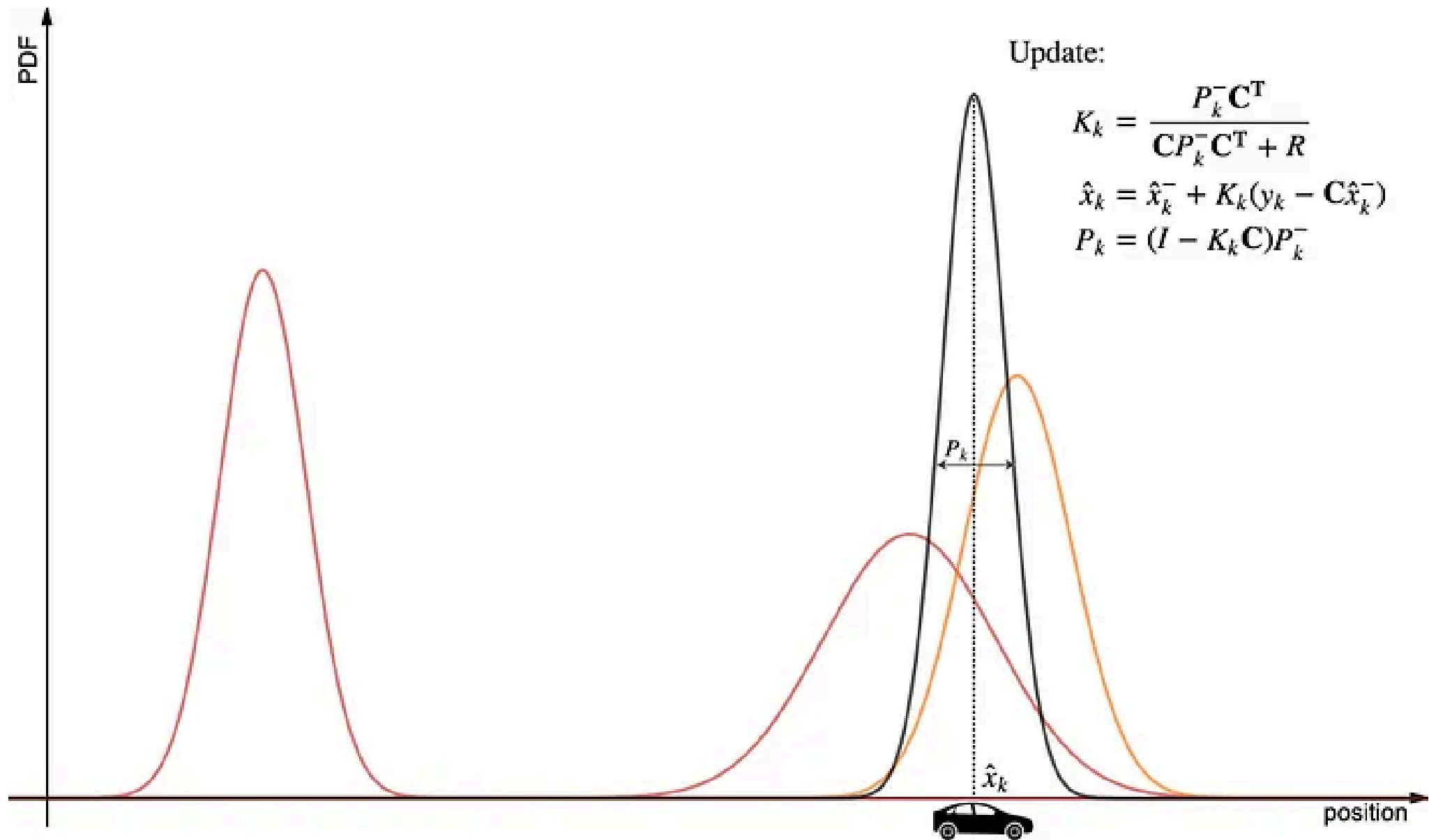
- P_k^- 는 K_k 비율만큼 줄어든다

$K_k = 0 \rightarrow P_k = (1-0) \cdot P_k^- = P_k^-$ (전혀 안 줄어듦 — 센서 무시)
 $K_k = 0.5 \rightarrow P_k = (1-0.5) \cdot P_k^- = 0.5P_k^-$ (절반으로 줄어듦)
 $K_k = 1 \rightarrow P_k = (1-1) \cdot P_k^- = 0$ (이론상 완전히 사라짐)

- $K_k = 1$ 이 되려면 $R = 0$ (완벽한 센서)이어야 함 — 현실에서 완벽한 센서는 없으므로 P_k 는 절대 완전히 0이 되지 **않**
음



Update Result



- 초기값 설정 (1회만)

변수	의미	설정 주체
$\hat{x}_0$	초기 위치·속도 추정값	사용자 설정
P_0	초기 불확실성	사용자 설정
A, B, C, Q, R	물리 모델·센서 모델	사용자 설계

- ① Predict — "다음에 어디 있을까?" (모델로 예측)

$$\hat{x}_k^- = A \cdot \hat{x}_{k-1} + B \cdot u_k \quad \leftarrow \text{운동 모델로 상태 예측}$$

$$P_k^- = A \cdot P_{k-1} \cdot A^T + Q \quad \leftarrow \text{불확실성 증가}$$

- ② Update — "센서가 뭐라 했지?" (실측값으로 보정)

$$K_k = P_k^- \cdot C^T \cdot (C \cdot P_k^- \cdot C^T + R)^{-1} \quad \leftarrow \text{예측 vs 센서 가중치}$$

$$\hat{x}_k = \hat{x}_k^- + K_k \cdot (y_k - C \cdot \hat{x}_k^-) \quad \leftarrow \text{중간값 탐색}$$

$$P_k = (I - K_k \cdot C) \cdot P_k^- \quad \leftarrow \text{불확실성 감소}$$

- 핵심: K_k 이 0에 가까울수록 예측을, 1에 가까울수록 센서를 더 신뢰
 - P_k^- 와 R 의 상대적 크기로 자동 결정 — 사람이 직접 정하지 않음
- ① → ② → ① → ② → ... 반복하며 수렴

핵심 행렬 한눈에 정리

행렬	이름	한 줄 비유	보통 고정?
A	상태 전이 행렬	"물리 법칙 (등속이면 저기 있겠지)"	✓ 고정
B	제어 행렬	"내가 내린 명령의 효과"	✓ 고정
P_k^- / P_k	추정 공분산	"내 추정이 얼마나 불확실한가"	✗ 매 스텝 변함
Q_k	프로세스 노이즈	"A로 못 담은 외란을 인정"	✓ 보통 고정
C	센서 행렬	"센서가 상태의 무엇을 재는가 (1/0)"	✓ 고정
R_k	센서 노이즈	"센서 신뢰도의 역수"	✓ 거의 고정
K_k	칼만 게인	" P_k^- 와 R로 자동 계산되는 가중치"	✗ 매 스텝 계산

• 칼만 필터 설계 우선순위

- 1순위 A — 현실 운동을 얼마나 잘 모델링했는가 (성능의 기반)
- 2순위 Q — 모델의 한계를 얼마나 솔직하게 인정했는가 (튜닝의 핵심)
- 3순위 R — 실험으로 측정 가능 (비교적 쉬움)
- 4순위 P_0 — 잘못 설정해도 자동 수렴 (영향 가장 적음)

- "왜 예측하면 오차가 더 커지나요?"
 - 예측은 새로운 정보 없이 과거 데이터만으로 추측하기 때문
 - 눈을 감고 걸으면 시간이 지날수록 자신의 위치를 확신하기 어려운 것과 동일
- "행렬(A, C, Q, R)들의 정체가 뭔가요?"
 - A: 물리 법칙(속도=거리/시간)을 담은 행렬 — 상태 전이
 - C: 센서가 측정 가능한 상태변수를 1, 불가능한 것을 0으로 표현
 - Q: "내 모델이 얼마나 불완전한지 인정하는 값" — 프로세스 노이즈
 - R: "센서 신뢰도의 역수" — 클수록 센서를 덜 믿음
- " K_k 가 1이 되면 P_k 가 0이 되는데 괜찮은가요?"
 - $K_k = 1$ 은 $R = 0$ (완벽한 센서)일 때만 가능
 - 현실에서 완벽한 센서는 없으므로 P_k 는 절대 완전히 0이 되지 않음
 - 칼만 필터는 항상 "완전한 확신 없이 최선의 추정" 을 하는 알고리즘

Part 6. 칼만 필터 YOLO에 적용하기

이 자료의 목표

이론에서 배운 것	코드에서 나오는 것
상태벡터 $\hat{x}$, 공분산 P	<code>self.x</code> , <code>self.P</code>
상태 전이 행렬 A	<code>self.A</code>
센서 행렬 C	<code>self.C</code>
프로세스 노이즈 Q	<code>self.Q</code>
센서 노이즈 R	<code>self.R</code>
Predict 수식	<code>def predict()</code>
Update 수식	<code>def update()</code>

- BBoxKalmanTracker 클래스의 4개 메서드

```
BBoxKalmanTracker
├── __init__()    ← 행렬들 초기화 (A, C, Q, R, P, x)
├── init()        ← 첫 탐지 시 초기 위치 설정
├── predict()     ← ① Predict 단계 구현 (TODO)
└── update()     ← ② Update 단계 구현 (TODO)
```

- 매 프레임마다 호출 순서

```
1. model.predict(frame)    YOLO로 사람 탐지 → cx, cy 얻기
2. tracker.predict()       칼만: 다음 위치 예측
3. tracker.update(cx, cy)  칼만: YOLO 결과로 보정
```

- 이 트래커는 사람의 중심점 (cx, cy)을 추적합니다

```
# 상태벡터: [cx, vx, cy, vy]
#           x방향 위치, x방향 속도, y방향 위치, y방향 속도
self.x = np.zeros((4, 1))
```

- 왜 속도(vx, vy)도 포함하나요?
 - 위치만 알면 "다음 프레임에 어디 있을지" 예측 불가.
 - 속도를 알면 → 다음 위치 = 현재 위치 + 속도 × 시간

인덱스	변수	의미
<code>x[0]</code>	<code>cx</code>	중심점 x 좌표
<code>x[1]</code>	<code>vx</code>	x 방향 속도 (픽셀/프레임)
<code>x[2]</code>	<code>cy</code>	중심점 y 좌표
<code>x[3]</code>	<code>vy</code>	y 방향 속도 (픽셀/프레임)

- "등속 운동"을 행렬로 표현한 것

$$\text{다음 위치} = \text{현재 위치} + \text{속도} \times \Delta t$$

```
self.A = np.array([
    [1, dt, 0, 0],      # cx_next = cx + vx*dt
    [0, 1, 0, 0],      # vx_next = vx (속도 불변)
    [0, 0, 1, dt],      # cy_next = cy + vy*dt
    [0, 0, 0, 1]        # vy_next = vy (속도 불변)
], dtype=float)
```

- 행렬 곱 $A @ x$ 를 직접 계산해보면

- `dt=1.0` → 1프레임 단위. 실제 시간(초)으로 하려면 `dt = 1/fps`

$$A\hat{x} = \begin{bmatrix} 1 & dt & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & dt \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} c_x \\ v_x \\ c_y \\ v_y \end{bmatrix} = \begin{bmatrix} c_x + v_x \cdot dt \\ v_x \\ c_y + v_y \cdot dt \\ v_y \end{bmatrix}$$

- "YOLO는 cx와 cy만 줍니다. vx, vy는 측정 불가!"
 - C 행렬 크기: 측정값 수 × 상태변수 수 = 2×4

```
# 상태벡터: [cx, vx, cy, vy]
# YOLO가 주는 것: [cx,      cy      ]

self.C = np.array([
    [1, 0, 0, 0], # cx만 측정
    [0, 0, 1, 0]  # cy만 측정
], dtype=float)
```

- C @ x 를 계산하면:

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} c_x \\ v_x \\ c_y \\ v_y \end{bmatrix} = \begin{bmatrix} c_x \\ c_y \end{bmatrix} \quad \leftarrow \text{YOLO가 제공하는 값과 동일!}$$

- Q, R, P의 초기값과 물리적 의미

```
self.Q = np.eye(4) * 0.1    # 프로세스 노이즈: 작게 → 모델을 신뢰
self.R = np.eye(2) * 50.0    # 센서 노이즈: 크게 → YOLO를 덜 신뢰
self.P = np.eye(4) * 500     # 초기 불확실성: 크게 → "처음엔 전혀 모름"
```

변수	값	의미
$Q = 0.1 \cdot I$	작음	"등속 모델을 꽤 믿는다"
$R = 50 \cdot I$	큼	"YOLO 탐지는 ± 50 픽셀 오차 있다"
$P = 500 \cdot I$	매우 큼	"처음엔 위치/속도를 전혀 모른다"

- $P_0 = 500$ 은 큰 값이지만 몇 프레임 후 자동으로 현실적인 값으로 수렴합니다.
잘못 설정해도 괜찮은 파라미터!

- 처음 사람을 탐지했을 때 딱 한 번 호출됩니다

```
def init(self, cx, cy):  
    self.x = np.array([[cx], [0.0], [cy], [0.0]], dtype=float)  
    self.initialized = True
```

```
self.x = [cx]    ← YOLO가 알려준 첫 위치  
         [0.0]  ← 속도는 0으로 가정 (처음엔 모름)  
         [cy]   ← YOLO가 알려준 첫 위치  
         [0.0]  ← 속도는 0으로 가정
```

- 속도를 0으로 초기화하는 이유: 첫 프레임에서는 이전 프레임이 없어 속도를 계산할 수 없기 때문.
몇 프레임 후 자동으로 실제 속도에 수렴!

- 수식:

$$\hat{x}_k^- = A\hat{x}_{k-1}$$

$$P_k^- = AP_{k-1}A^T + Q$$

- Python 코드로 변환:

```
def predict(self):  
    # ① 상태 예측:  $x = A @ x$   
    self.x = self.A @ self.x  
  
    # ② 공분산 예측:  $P = A @ P @ A.T + Q$   
    self.P = self.A @ self.P @ self.A.T + self.Q  
  
    # ③ 예측된  $cx$ ,  $cy$  반환  
    return self.x[0, 0], self.x[2, 0]
```

- 코드 각 줄의 의미

- `self.x = self.A @ self.x`

- "A 행렬(물리 법칙)을 현재 상태에 곱해서 다음 상태를 예측"
- $c_{x,next} = c_x + v_x \cdot \Delta t$, $c_{y,next} = c_y + v_y \cdot \Delta t$ 계산

- `self.P = self.A @ self.P @ self.A.T + self.Q`

- "예측하면 불확실성이 커진다 (Q만큼 증가)"
- 눈 감고 걸으면 자기 위치 확신이 줄어드는 것과 동일

- `return self.x[0, 0], self.x[2, 0]`

- `self.x` 는 (4,1) 행렬
- `self.x[0, 0]` → cx (x 좌표), `self.x[2, 0]` → cy (y 좌표)

- 수식:

$$K_k = P_k^- C^T (C P_k^- C^T + R)^{-1}$$

$$\hat{x}_k = \hat{x}_k^- + K_k (y_k - C \hat{x}_k^-)$$

$$P_k = (I - K_k C) P_k^-$$

```
def update(self, cx, cy):
    z = np.array([[cx], [cy]], dtype=float)      # 측정값

    innovation = z - self.C @ self.x             # 예측 vs 실측 차이
    S = self.C @ self.P @ self.C.T + self.R      # Innovation 공분산
    K = self.P @ self.C.T @ np.linalg.inv(S)     # 칼만 게인

    self.x = self.x + K @ innovation              # 상태 보정
    self.P = (np.eye(4) - K @ self.C) @ self.P   # 공분산 보정

    return self.x[0, 0], self.x[2, 0]
```

- 코드 각 줄의 의미 (측정값 → Innovation)

- `z = np.array([[cx], [cy]], dtype=float)`

- YOLO가 알려준 실제 측정값 (2×1 벡터)

- `innovation = z - self.C @ self.x`

- **Innovation = 실제 측정값 - 내 모델의 예측 측정값**
 - "YOLO가 말한 위치" - "내가 예측한 위치" = 얼마나 빗나갔나?

- `S = self.C @ self.P @ self.C.T + self.R`

- Innovation의 불확실성 = 내 예측 불확실성 + 센서 불확실성

- Innovation이 크면 → 많이 빗나감 → 센서를 더 믿어야

- Innovation이 작으면 → 잘 맞음 → 예측을 더 믿어도 됨

- 코드 각 줄의 의미 (칼만 게인 → 보정)

- `K = self.P @ self.C.T @ np.linalg.inv(S)`
 - 칼만 게인 계산 — "예측 vs 센서, 무엇을 얼마나 믿을까?"
 - `np.linalg.inv(S)` = S^{-1} (역행렬)
- `self.x = self.x + K @ innovation`
 - 상태 보정 — K만큼 Innovation을 반영
 - K=0이면 예측값 그대로, K=1이면 센서값으로 완전 대체
- `self.P = (np.eye(4) - K @ self.C) @ self.P`
 - 불확실성 감소 — 센서 정보를 받았으니 불확실성이 줄어듦
 - `np.eye(4)` = 4×4 단위행렬 I

- 수학 수식과 Python 코드 1:1 대응

수학 기호	Python 코드	의미
$A\hat{x}$	<code>self.A @ self.x</code>	상태 예측
APA^T	<code>self.A @ self.P @ self.A.T</code>	공분산 예측
$y - C\hat{x}^-$	<code>z - self.C @ self.x</code>	Innovation
$CP^-C^T + R$	<code>self.C @ self.P @ self.C.T + self.R</code>	Innovation 공분산
$(\dots)^{-1}$	<code>np.linalg.inv(...)</code>	역행렬
K_k	<code>K</code>	칼만 게인
I	<code>np.eye(4)</code>	단위행렬
$(I - KC)P^-$	<code>(np.eye(4) - K @ self.C) @ self.P</code>	공분산 보정

- 매 프레임마다 YOLO + 칼만 필터의 연동 순서

```
while True:
    success, frame = cap.read()          # 웹캠 프레임 읽기

    results = model.predict(frame, ...)  # YOLO: 사람 탐지

    best_box = max(r.bboxes, ...)         # 신뢰도 가장 높은 1명 선택
    cx, cy = best_box.xywh[0][:2]        # 바운딩박스 중심점 추출

    # 칼만 필터 적용
    if not tracker.initialized:
        tracker.init(cx, cy)             # 최초 1회: 초기화

    tracker.predict()                     # ① 예측
    filtered_cx, filtered_cy = tracker.update(cx, cy)  # ② 보정

    # 시각화
    # 빨간 점 = YOLO 원본 (떨림 있음)
    # 파란 점 = 칼만 보정 (부드러움)
```

- YOLO → cx, cy 제공 → 칼만 update() 입력 → 부드러운 filtered_cx, filtered_cy 출력

핵심 파라미터 튜닝 가이드

- Q, R 파라미터 조절에 따른 필터 동작 변화

```
self.Q = np.eye(4) * 0.1    # 이 값을 올리면?  
self.R = np.eye(2) * 50.0  # 이 값을 올리면?
```

파라미터	올리면	내리면
Q 증가	모델을 덜 믿음 → 센서 반응 빠름, 떨림 증가	모델을 더 믿음 → 부드럽지만 느린 반응
R 증가	YOLO를 덜 믿음 → 더 부드럽지만 느림	YOLO를 더 믿음 → 빠른 반응, 떨림 증가

- 실험 과제

- Q = 10.0으로 올려보기 → 더 빠르게 반응하지만 떨림 증가
- R = 5.0으로 내려보기 → YOLO 값에 더 민감하게 반응
- USE_KALMAN = False → 칼만 OFF시 얼마나 떨리는지 비교

- ① Predict: 물리 법칙으로 예측 (불확실성 증가)

```
self.x = self.A @ self.x
self.P = self.A @ self.P @ self.A.T + self.Q
```

- ② Update: 센서로 보정 (불확실성 감소)

```
K = self.P @ self.C.T @ np.linalg.inv(self.C @ self.P @ self.C.T + self.R)
self.x = self.x + K @ (z - self.C @ self.x)
self.P = (np.eye(4) - K @ self.C) @ self.P
```

- ③ K가 핵심: 예측 vs 센서 신뢰도 균형

K → 0 : 예측을 신뢰 (센서 노이즈 R이 클 때)
K → 1 : 센서를 신뢰 (예측 불확실성 P가 클 때)

- 이 두 단계를 매 프레임 반복하면 → 부드럽고 안정적인 추적

Part 7. 칼만 필터 성능 평가

칼만 필터 평가 — 시계열과 무엇이 다른가?

- 시계열 모델 (ARIMA, LSTM 등)의 평가
 - 모델이 출력하는 것: 예측값 $\hat{y}$ 하나
 - 평가 기준: RMSE, MAE 등으로 정답과의 거리 측정
- 칼만 필터가 출력하는 것 — $\hat{x}_k$ 와 P_k
 - $\hat{x}_k$: 상태 추정값 (예측값)
 - P_k : 추정 오차 공분산 (자신의 불확실성)
 - 칼만 필터는 예측값 **AND** 자신의 불확실성을 함께 출력 → 평가 대상이 두 가지다
- 올바른 평가 기준 — 일관성(Consistency)
 - 단순 기준: "분산이 작고 예측값을 잘 맞추면 좋은 필터"
 - 올바른 기준: 실제 오차의 범위를 **정직하게** 표현하면서 예측값도 잘 맞추는가?
 - 일관성: 필터가 자신의 불확실성을 얼마나 정확히 표현하는가

- 필터 일관성 평가 기준

상황	필터의 주장 (P)	실제 오차	판정
과신 (Overconfident)	매우 작음 — "99% 확신"	크다	위험
과소신 (Underconfident)	너무 큼 — "50%만 확신"	작다	비효율
일관성 있음	적절함	P 에 부합	정상

- 과신 필터의 위험성

- 칼만 이득 K 가 작아짐 → 측정값을 무시 → 발산 위험

- RMSE (Root Mean Square Error): 추정값과 참값의 차이를 수치화하는 가장 기본적인 지표

$$\text{RMSE} = \sqrt{\frac{1}{N} \sum_{k=1}^N \|\hat{x}_k - x_k^{true}\|^2}$$

- 특성 정리

항목	내용
장점	직관적, 범용적, 다른 필터와 비교 용이
단점	참값(ground truth)이 반드시 필요
적용 환경	시뮬레이션 (참값이 있을 때)
활용	위치/속도 추적 오차 평가

- **MAE (Mean Absolute Error):** 이상치(outlier)에 덜 민감한 대안적 오차 지표

$$\text{MAE} = \frac{1}{N} \sum_{k=1}^N |\hat{x}_k - x_k^{true}|$$

- **선택 기준**

- **RMSE 선호 상황**

- 큰 오차에 더 민감하게 반응하고 싶을 때
- 분산 분석과 연계할 때

- **MAE 선호 상황**

- 이상치가 많은 실측 데이터
- 오차의 평균적 크기가 중요할 때

- NEES (Normalized Estimation Error Squared): 추정 오차가 공분산 P 와 일관성이 있는가?

$$\epsilon_x(k) = \tilde{x}_k^T P_k^{-1} \tilde{x}_k, \quad \tilde{x}_k = x_k^{true} - \hat{x}_k, \quad \epsilon_x(k) \sim \chi^2(n_x)$$

- 해석 기준 (n_x : 상태벡터의 차원, 자유도)

평균 NEES	의미	원인
$\approx n_x$	정상 (일관성 있음)	필터 잘 튜닝됨
$\ll n_x$	과소신 (Underconfident)	Q, R 과대 추정
$\gg n_x$	과신 (Overconfident)	Q, R 과소 추정

- 제약사항: 참값(x^{true})이 필요 $\rightarrow$ 시뮬레이션 환경에서만 사용 가능
 - 여러 Monte Carlo 실행의 평균 NEES $\approx n_x \rightarrow$ 올바르게 튜닝된 필터

- NIS (Normalized Innovation Squared): 이노베이션이 이노베이션 공분산 S 와 일관성이 있는가?

$$\epsilon_{\nu}(k) = \nu(k)^T S(k)^{-1} \nu(k), \quad \nu(k) = y(k) - \hat{y}(k), \quad S = (C P_k^- C^T + R_k), \quad \epsilon_{\nu}(k) \sim \chi^2(n_z)$$

- NIS의 핵심 장점: 참값 없이도 실제 환경에서 사용 가능
 - $\nu(k) = y(k) - \hat{y}(k) \rightarrow$ 실제 측정값과 예측 측정값만 있으면 됨
 - R 파라미터 튜닝의 핵심 도구
- 해석 기준 (n_z : 측정벡터의 차원, 자유도)

평균 NIS	의미
$\approx n_z$	이노베이션 공분산 일관성 정상
$\gg n_z$	측정 노이즈 R 과소 추정
$\ll n_z$	측정 노이즈 R 과대 추정

- 두 일관성 지표 비교

구분	NEES	NIS
비교 대상	추정 오차 vs P	이노베이션 vs S
참값 필요 여부	필요	불필요
적용 환경	시뮬레이션	실제 환경 가능
분석 공간	전체 상태공간	측정 공간만
자유도	n_x (상태 차원)	n_z (측정 차원)
주요 용도	디버깅, 심층 분석	실시간 모니터링, R 튜닝

- 실전 권장 전략

- 시뮬레이션: NEES + NIS 모두 검사 → 실제 환경: NIS만 활용

- Innovation = 측정값 - 예측 측정값 = $\nu(k) = y(k) - \hat{y}(k)$
 - 필터가 올바르게 동작한다면 Innovation은 **White Noise**여야 한다
- 검사 항목과 판정 기준

검사 항목	기대 결과	위반 시 의미
평균	≈ 0	체계적 편향 존재 (모델 오류)
분포	가우시안 (정규분포)	비선형성 누락
자기상관	비상관 (White Noise)	모델이 정보를 다 활용 못함
크기	$S(k)$ 범위 내	Q 또는 R 파라미터 오류



- 증상별 원인 파악 및 처방

증상	원인	처방
RMSE 크고 NEES $\gg n_x$	Q 너무 작음	Q 증가
RMSE 크고 NEES $\ll n_x$	R 너무 작음	R 증가
NIS $\gg n_z$	R 과소 추정	R 증가
NIS $\ll n_z$	R 과대 추정	R 감소
Innovation 자기상관	모델 불완전	상태/모델 재설계
P 발산	수치 불안정	Joseph form 사용